

Engineering Intelligence Hub

A code-RAG system over Apache Airflow — six weeks of measured iteration.
Project analysis, technical journey, and recruiter Q&A reference.

Author: Hrishav Banerjee

Repository: github.com/hrishav11/Engineering-intelligence-hub-RAG

Part 1 — The Journey, Week by Week

The project was built in eight weekly milestones, each ending in a runnable, measurable deliverable. What follows is the honest record — what shipped, what didn't, and what each week taught.

Week 1 — Naive RAG **[Done]**

Scaffolded the project: Python CLI, Chroma vector DB, OpenAI embeddings (text-embedding-3-small), Claude Haiku 4.5 as the generator. Walked the Airflow repo, extracted markdown docs + Python docstrings, chunked by tokens (~500 each), embedded, stored, retrieved top-k for any question, and asked Claude to answer with [#N] citations. End of week: a CLI that could answer 'how does the scheduler work?' with real citations.

Week 2 — Code-aware chunking + hybrid retrieval **[Done]**

Replaced naive token chunking with tree-sitter parsing — each Python function and class became its own chunk, including the body, not just the docstring. Added BM25 keyword retrieval alongside vector search, fused using Reciprocal Rank Fusion. Identifier-aware tokenization split snake_case and CamelCase so queries like 'GCSHook.upload' could match the actual method names. This was the largest single jump in quality.

Week 2.5 — Symbol injection (mid-week fix) **[Done]**

Discovered that BM25 was ranking the class header chunk above the actual method even when the query named the method exactly (e.g. 'GCSHook.upload' returned the class definition first). Added a symbol-pinning layer: if a query contains an exact Class.method identifier, those chunks get pinned to the top of results, bypassing the RRF score. Targeted fix, measurable improvement on identifier-heavy queries.

Week 3 — Evaluation harness **[Done]**

Built the eval infrastructure: 17 hand-curated questions with expected file paths and symbols per question, retrieval metrics (hit@k, MRR), and LLM-as-judge scoring on a 0-3 rubric using Claude. Saved every run as a timestamped JSON baseline so future changes are comparable. This shifted the project from vibes-based to measurement-based development — every later week was measured against this scorecard.

Week 4 — Cross-encoder rerank + HyDE [Mixed (later fixed)]

Added two well-known RAG improvements: the bge-reranker-base cross-encoder (re-scores top-30 candidates) and HyDE (Claude writes a hypothetical answer, embed that for vector search). Both built cleanly. Both initially measured as worse than plain hybrid on the eval set. This was an important honest finding: open-source rerankers trained on web search don't transfer well to code. Later (Week 8) added Voyage AI's code-trained rerank-2-lite, which DID improve strict hit@1 by +28%.

Week 4.5 — Question router [Done]

Added a heuristic regex classifier that picks the retrieval method per question: conceptual phrases -> HyDE, code identifiers -> hybrid, multi-hop signals -> agentic. Cheap, debuggable, and avoided one-size-fits-all retrieval. In practice the router only fires non-default paths on ~25% of questions, which is enough to matter when it does (especially on multi-hop).

Week 5 — GitHub issues + PRs [Done]

Ingested closed GitHub issues and PRs from apache/airflow as additional chunks (same Chroma collection, synthetic source paths like `_gh/airflow/issues/12345`, GitHub URLs reconstructed at render time). Hit the GitHub anonymous API's 1000-result deep-pagination cap; documented as a known limit. Cross-corpus retrieval now works — questions like 'have there been recent issues with X?' surface real issue threads alongside code.

Week 6 — Agentic retrieval [Mixed (later fixed)]

Gave Claude a search tool it could call multiple times. The agent issues a query, reads the chunks, decides what to search for next, repeats up to 5 turns, then synthesizes an answer. Qualitatively transformative on multi-hop questions ('when X fails, what callbacks fire and in what order?') — but the single-shot hit@k metric couldn't measure its value because it accumulates 40+ chunks across multiple searches. Fixed in Week 7 by adding `path_coverage` as a new metric.

Week 6.5 — Better evaluation infrastructure [Done]

Three measurement upgrades: (1) auto-generated 344 eval questions from random chunks (vs 17 hand-curated — 20x more statistical power); (2) hand-wrote 21 multi-hop questions that genuinely span multiple files; (3) added the `path_coverage` and `symbol_coverage` metrics that measure 'did the right file appear ANYWHERE in retrieved chunks' rather than just 'in the first k'. This is the metric agentic actually needed.

Week 7 — Web UI + intelligent fallback [Done]

Built a Streamlit UI with method selector, transparent routing display, and clickable GitHub citations. Light/dark theme toggle, industry-grade design with orange accents. Added the most important runtime improvement of the project: an `INSUFFICIENT_CONTEXT` sentinel — when the model judges its retrieved chunks insufficient to answer, it emits a sentinel token and the system transparently re-runs with agentic retrieval. Cheaper than always-agentic, smarter than upfront classification.

Week 8 — Voyage rerank + honest README + final eval [Done]

Added Voyage AI's rerank-2-lite (code-trained) as a method. Ran the definitive 200-question x 4-method eval. Voyage improved strict hit@1 by +28% — vindicating Week 4's reranker attempt with a domain-appropriate model. Agentic via routed-with-fallback measured 3.2x better path coverage on multi-hop. Wrote an honest README documenting every number, every dead end, and every win.

Part 2 — Final Technical State

Architecture

Every question flows through this pipeline:

1. **Router** — heuristic regex classifier categorizes the question: code / conceptual / multi-hop.
2. **Retrieval method dispatch** — code -> hybrid, conceptual -> hybrid_hyde, multi-hop -> agentic.
3. **Hybrid retrieval** — BM25 (keyword) + vector (cosine on OpenAI embeddings) + reciprocal rank fusion + symbol pinning.
4. **Answer generation** — Claude Haiku 4.5 reads the retrieved chunks and writes an answer with [#N] citations. If the chunks are insufficient, emits a sentinel that triggers automatic escalation to agentic retrieval.
5. **Model fallback** — on Anthropic API errors (rate limit, 5xx), automatically falls back to Sonnet 4.6.

Final measured numbers

200-question eval x 4 methods. Two metric tiers: path-only (did the right file surface?) and strict (did the exact expected chunk surface?).

Path-only metrics

Method	hit@1	hit@3	hit@10	path_cov	judge
routed (default)	0.555	0.735	0.880	0.868	1.80
hybrid	0.535	0.710	0.845	0.816	1.80
hybrid_voyage_rerank	0.545	0.725	0.845	0.816	1.78
hybrid_rerank (bge)	0.568	0.724	0.899	0.869	1.79

Strict metrics — exact chunk match

Method	hit@1	hit@3	hit@10	sym_cov
routed (default)	0.195	0.450	0.660	0.715
hybrid	0.180	0.440	0.630	0.642
hybrid_voyage_rerank	0.230 *	0.495 *	0.630	0.642
hybrid_rerank (bge)	0.181	0.407	0.673	0.675

Multi-hop only (14 questions)

Method	p_hit@3	path_cov
routed (w/ fallback)	0.57 *	0.45 *
hybrid	0.29	0.14
hybrid_voyage_rerank	0.07	0.14

hybrid_rerank (bge)	0.07	0.18
---------------------	------	------

(* = best in column)

Honest assessment

Above-average for a personal RAG project, mostly because it has an honest eval. Not production-quality: no real-user testing, single corpus (Airflow), single language (Python chunker), eval set of 379 questions (production teams use thousands), auto-generated questions have a known answer chunk by construction. The journey — measuring six weeks of changes, finding three techniques that didn't work, fixing two of them later — is the real artifact.

Part 3 — Recruiter Q&A Reference (40 Questions)

Likely questions if a recruiter or interviewer sees this project on your resume, with prepared answers. Grouped by theme. Use these as preparation — paraphrase in your own voice during the interview, don't memorize verbatim.

Section A — Project Overview & Motivation

1. In one sentence, what is this project?

A retrieval-augmented generation system that answers questions about the Apache Airflow codebase by retrieving from 65,000 indexed chunks of code, docs, and GitHub issues, and citing real files in its answers.

2. Why did you build this?

Two reasons. First, to learn RAG hands-on — I wanted a project where every retrieval improvement could be measured, not just claimed. Second, because hiring managers in engineering roles are themselves engineers — they'll actually try the demo and read the code, which is rare for portfolio projects.

3. Why Apache Airflow specifically?

It's a large, well-known codebase (~7,000 Python files) with rich documentation and an active GitHub history. Three things together: I can evaluate answer quality myself because I understand the domain, the corpus is non-trivially large, and the issues/PRs give me cross-corpus retrieval to demonstrate.

4. What problem does it solve that a regular Claude or ChatGPT call doesn't?

Out-of-the-box LLMs answer from training data — they don't know the specific version of Airflow you're running, they can't cite real files, and they'll happily hallucinate function signatures. RAG grounds the answer in actual retrieved chunks and forces citations, so every claim is checkable.

5. Who would use it?

Realistically: nobody right now — it's a portfolio project. Conceptually: new engineers onboarding to a codebase, or experienced ones investigating an area they don't usually touch. The 'engineering intelligence hub' framing is honest about the target use case even though I haven't tested with real users.

6. How long did it take?

Eight weekly milestones — roughly six weeks of focused work with breaks. Each week added a measurable capability. I tracked progress against an eval harness from Week 3 onward, so 'progress' meant 'numbers moved', not 'I added code'.

7. What's the most impressive thing about the project, in your opinion?

Not any single technique. It's that I measured every change, found several techniques that didn't work, and documented them honestly in the README. The eval discipline is what separates this from typical 'I built a RAG' projects.

8. What did you learn that surprised you?

Three things. First, open-source rerankers trained on web search hurt code RAG — they prefer test files over implementations. Second, my single-shot hit@k metric was actively misleading on multi-hop questions; the metric was the bug, not the technique. Third, simple symbol pinning (a 20-line regex matcher) beat several more complex techniques I added.

9. What would you do differently if you started over?

Build the eval harness first, not in Week 3. The early weeks were vibes-based development — I'd add a technique, think it helped, and move on. Once the eval existed, I could see that some of those 'improvements' had actually regressed quality. I'd also write 200+ eval questions upfront instead of 17.

10. How much did it cost to build?

Under \$20 in API costs total. OpenAI embeddings for ingestion (~\$1 once), Claude calls for hundreds of eval runs (~\$15), and Voyage AI rerank credits (free tier was enough). Most of the cost was Claude judge calls during eval.

Section B — Technical Architecture

11. Explain RAG in your own words.

Retrieval-Augmented Generation. Two parts: retrieval finds the most relevant text from a knowledge source, augmentation stuffs that text into the LLM prompt as context, generation has the LLM read the question + context and write an answer. The hard part is retrieval — generation is almost trivial once you have good chunks.

12. What's the difference between vector search and BM25?

Vector search measures semantic similarity in embedding space — 'how upload a file to GCS' might match a chunk about cloud storage transfers because they're conceptually related. BM25 is keyword frequency with smart weighting — it requires literal token overlap. They make complementary mistakes, so combining them via reciprocal rank fusion outperforms either alone.

13. What's Reciprocal Rank Fusion (RRF)?

A way to combine multiple ranked lists into one. Each chunk gets a score = sum of $1/(K + \text{rank})$ across all lists it appears in. K is a constant (60 by convention) that controls how much rank position matters versus appearing in multiple lists. Chunks ranked high in BOTH BM25 and vector beat chunks ranked very high in only one.

14. How does tree-sitter chunking work?

Tree-sitter parses code into an AST. I walk the AST and emit one chunk per function, class header, and method, with full bodies. Compared to fixed-size token windows, this means a method's docstring AND its implementation are in the same chunk — so 'how does X work?' can find both.

15. What's symbol pinning?

When a query contains an exact identifier like 'GCSHook.upload' or 'BaseXCom.serialize_value', the system pins chunks whose symbol metadata matches exactly to the top of the result, bypassing BM25 ranking. Solves the specific failure where BM25 ranks the class header above the actual method even when the user named the method explicitly.

16. What's HyDE? Why did it underperform?

Hypothetical Document Embeddings — instead of embedding the question, you ask the LLM to write a hypothetical answer first, then embed that. The hypothetical is more similar to actual answer chunks than the question is. It underperformed because the hypothetical sometimes names wrong identifiers, which pull retrieval in the wrong direction. Slightly worse than plain hybrid on this eval.

17. What's a cross-encoder reranker?

A model that reads (query, chunk) pairs together and outputs a relevance score. Slower than vector similarity but more accurate because it sees both inputs at once. bge-reranker-base (web-trained) hurt my numbers; Voyage AI's rerank-2-lite (code-trained) improved strict hit@1 by 28%. Lesson: domain training matters.

18. How does agentic retrieval work here?

Claude gets a search tool. It can call search with any query, read the returned chunks, then decide to search again with a refined query. Up to 5 turns per question. After exploring, it synthesizes a final answer using everything it saw. Powerful for multi-hop questions ('when X happens, what fires in what order?').

19. What's the INSUFFICIENT_CONTEXT sentinel?

A runtime escalation pattern. The system prompt instructs Claude to emit a sentinel token if its retrieved chunks don't actually answer the question. The downstream system detects the sentinel and re-runs with agentic retrieval. Cheaper than always-agentic (~80% of questions don't need it), smarter than upfront classification (detects actual failure, not predicted failure).

20. How do GitHub issues fit into the same corpus as code?

Each issue and PR becomes a chunk with a synthetic source path like '_gh/airflow/issues/12345'. Same Chroma collection, same embeddings model, same BM25 index. At display time, those synthetic paths are reconstructed into real GitHub URLs. Cross-corpus retrieval works automatically — a question about a bug can surface both the code AND the issue thread discussing it.

Section C — Evaluation & Measurement

21. How did you evaluate the system?

I built an eval harness with 379 questions, each tagged with expected file paths and symbols. For each (question, method), I run retrieval, generate an answer, and score: retrieval metrics (hit@k, MRR, path coverage, symbol coverage) and answer quality (Claude as judge on a 0-3 rubric). Save every run as a JSON baseline so I can diff between weeks.

22. What's hit@k? What's MRR?

Hit@k = did the right chunk appear in the top k results (0 or 1). MRR = mean reciprocal rank of the first correct chunk, averaged across questions. Both measure retrieval, not answer quality. They're standard IR metrics, but my big learning was that they don't measure agentic retrieval well — see path_coverage.

23. What's path_coverage and why did you add it?

Fraction of expected files matched ANYWHERE in cumulative retrieved chunks, not just in the first k. For agentic retrieval that accumulates 40+ chunks across multiple search calls, hit@k undersells it because the chunks beyond rank k might cover an expected path. path_coverage measures 'did the system eventually find the right files?', which is what matters for multi-hop questions.

24. Why is your strict hit@1 only 14-23%?

Because the 'strict' metric requires the EXACT expected chunk at rank 1 — not just any chunk from the right file. Many auto-generated eval questions name one specific chunk (e.g. 'GCSHook.upload' line 508-622) when several chunks from the same file would equally answer the question. Path-level hit@1 is 55%, which is what users actually experience.

25. How did you use Claude as a judge?

Separate Claude call per (question, answer) pair. System prompt: a rubric scoring 0-3 — 0 = wrong or hallucinated, 1 = vague but not wrong, 2 = mostly correct but partial, 3 = fully correct with specific citations. Returns JSON with score and one-line reason. Imperfect (variance $\sim \pm 0.08$ between runs) but consistent enough to compare methods.

26. How big is your eval set? Is that enough?

379 questions — 17 hand-curated, 21 hand-written multi-hop, 344 auto-generated. Honestly: small. Production RAG teams eval on thousands. With this size, retrieval-metric differences of less than 2% are statistical noise. Differences of 5%+ are real. Big enough to make the decisions I made, not big enough for confident publication-grade claims.

27. Why did you auto-generate questions instead of writing them all?

Hand-writing 1000 questions with expected paths takes weeks. Auto-generation: sample a chunk from the corpus, ask Claude to write a realistic user question this chunk would answer, set expected_paths = [that chunk's source path]. 30 minutes, \$1. Trade-off: every question has a known answer chunk, which makes retrieval somewhat easier than real user queries — but it's 20x more questions than I could hand-curate.

28. Walk me through one experiment where the measurement changed your decision.

I built bge-reranker-base in Week 4 expecting it to improve everything. Eval showed it hurt hit@3 by 24 points. Kept the code as experimental, removed it from the default. Later (Week 8), I tried Voyage AI's code-trained reranker and that DID help (+28% strict hit@1). Without the eval, I'd have shipped bge thinking it was an improvement.

29. What's a known limitation of your evaluation approach?

LLM-as-judge has run-to-run variance — the same answer can score 2 one run and 3 the next. I mitigate by running multiple runs and reporting mean +/- stddev, but the variance can be similar in size to the differences between methods. For confident claims you need 3+ runs and 200+ questions per category, which I'm short of.

30. If you had more time for eval, what would you do?

Three things. Expand to 1000+ questions with stratified sampling across categories. Run each (question, method) 3x and compute confidence intervals. Add real user queries scraped from the Airflow Stack Overflow tag, with expected_paths manually labeled. That last one is the most labor-intensive but produces the most trustworthy signal because the questions are authentic.

Section D — Decisions, Tradeoffs, Production Concerns

31. Why Claude Haiku and not Sonnet or GPT-4?

Cost. Haiku is ~\$1/M input tokens vs Sonnet's \$3/M, and for this RAG workload (where Claude reads chunks and writes an answer with citations) Haiku is plenty. I'd switch to Sonnet for harder reasoning tasks. The codebase has an automatic Haiku -> Sonnet fallback on errors via `call_with_fallback()`.

32. Why Chroma and not Pinecone, Weaviate, or pgvector?

Chroma runs locally with no infrastructure — fits a portfolio project. Persistent client writes to disk so the 65K embeddings survive restarts. For production with multi-tenancy or large scale I'd consider pgvector (already have Postgres) or Pinecone (managed). The interface is simple enough that swapping is one file change.

33. Why OpenAI embeddings and not local models?

text-embedding-3-small at \$0.02/1M tokens is essentially free and high quality. For a portfolio I'd rather not pay GPU costs to run a slightly-worse local model. If the project had privacy constraints I'd use a local model like bge-small-en or nomic-embed.

34. What's the cost per query?

Around \$0.005 for a simple hybrid query (OpenAI embed for the query + Claude Haiku answer). Around \$0.025 for an agentic query (5 Claude turns plus embeddings). Routed averages ~\$0.009 because only ~20% of questions escalate to agentic via the sentinel fallback.

35. How would you make this production-ready?

Five things, in order: (1) caching — same question shouldn't pay Claude twice; (2) authentication + per-user rate limits — anyone can rack up bills; (3) observability — log every (question, retrieval, answer, latency) tuple for analysis; (4) test the eval set against actual user logs to validate question distribution; (5) periodic re-ingest with date-range pagination for GitHub (currently capped at 1000 due to anonymous API limits).

36. What are the security concerns?

Three. First, prompt injection from the corpus — a malicious docstring could try to override the system prompt. I mitigate by isolating retrieved chunks in a clearly-delimited Context section, but it's not foolproof.

Second, API key exposure — keys are in .env (gitignored) and the README warns about it. Third, cost denial-of-service — no rate limits on the demo, so a public deployment could be exploited.

37. Why didn't you deploy the Streamlit app publicly?

The Chroma database is ~500 MB and gitignored, so any deploy needs to either re-ingest on startup (slow and costs money) or ship the DB to the host. Plus any public deployment runs up MY API bills per visitor. For a portfolio, the GitHub repo plus a Loom video are higher signal at lower risk than a possibly-broken public demo.

38. What's the biggest weakness of your project?

The eval set is too small for confident claims about small-percentage differences between methods. I scaled from 17 hand-written to 379 mixed, but production-grade comparison needs thousands. Secondary weakness: I never tested with real human users — the questions I generated and the answers I judged are all my own definitions of quality.

39. If you got 10 more hours on this, what would you do?

Run a 1000-question eval with 3 runs per method to get real confidence intervals. Write a fix for the GitHub pagination cap (date-range slicing). Ingest PR file-diffs and issue comments — that's where the actual diagnostic information lives. Then write a more polished portfolio piece around the most interesting finding (the bge vs Voyage rerank comparison) — that's genuinely publishable.

40. What's the single thing you're most proud of?

The honesty. I built techniques that didn't work, measured that they didn't work, kept them in the code as experimental, and documented the failure in the README. Most portfolio RAG projects hide their failures. Mine is the rare project where the failed attempts are part of the story, and one of them (the reranker) later turned into a win when I tried a better model.